IT314: Software Engineering (Mid-Evaluation Project Report)

Prof. Saurabh Tiwari

Mentor: Dhruvil Mehta

Project Title: Resume Analyser for College Placements

Upload it, understand it, improve it, get placed!

Group : 26

GitHub Repository:

https://github.com/Tanvii13/Resume-Analyzer-for-College-Placements

Name	Student ID
Sujal Balva	202401216
Tanishk Dhawan	202401224
Topaliya Shlok Ashokbhai	202401229
Trivedi Atharv	202401230
Vamja Krish Pravinbhai	202401239
Nakum Tanvi Dhiren	202401262
Solanki Taniskumar Mehulbhai	202401264
Tejot Harshil Rajendrabhai	202401265
Krish Hitendrakumar Paghadar	202401429
Prerak Patel	202401452

1. Concept Poster

GROUP 26

CONCEPT POSTER

Resume Analyzer for College Placements

Upload it, understand it, improve it, get placed!

Who is it for?

- Students preparing for placements and internships
- Placement officers and faculty / career counsellors
- Recruiters (for relevant skill matches)

What problem does it solve?

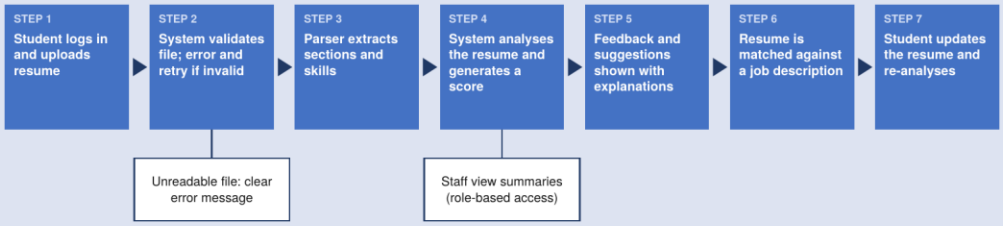
Students don't know how strong their resume is, which skills are missing, or how well it fits a specific job. Templates only fix formatting, and placement staff can't review every resume individually.

What is the big idea?

We use **NLP** to extract skills from a resume, **score it with clear explanations**, suggest improvements, and compare it against a job description. The system shows **matching and missing skills**, so students know exactly what to fix. Scores support human decisions and never replace them.

Illustrate how it works

- Student logs in and uploads a resume (PDF/DOCX)
- System validates the file and extracts education, skills, projects and experience
- Resume is analysed and given an overall score with reasons
- Strengths, weak areas and improvement suggestions are shown
- Resume is compared with a job description (matching vs. missing skills)
- Student improves the resume and re-analyses
- Placement staff view summaries (role-based access)



Why might it fail?

- Parsing may be inaccurate for unusual resume layouts or scanned files
- Scores may seem unfair or unclear if reasons aren't explained
- Privacy concerns, since resumes hold personal data
- Over-reliance on the score as the only measure of a candidate

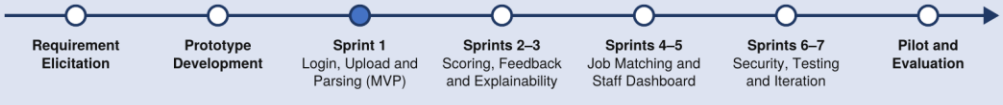
What should we prototype & test?

A working **upload, parse and display flow (Sprint 1)**, then an interactive interface to test with a small group of students and counsellors. We'll check whether the extracted information is correct and whether the feedback is understandable and useful.

How might we measure success?

- Parsing accuracy on manually checked sample resumes
- Correct handling of invalid, empty and corrupted files
- Relevance of matched and missing skills
- Students rating the feedback as clear and helpful
- Unauthorized users blocked from restricted data

How will we make this happen?



2. Project Overview

2.1 Introduction

The **Resume Analyser for College Placements** is a web-based tool that helps college students prepare for placements and internships. It analyses resume formatting and skill relevance, compares the student's profile with job and internship requirements, and gives feedback and practical suggestions. Placement officers and career counsellors can be supported with summaries, with access controlled by user role.

2.2 Problem Statement

Students find it difficult to judge the quality of their resumes, know what to improve, or see how well their skills match a particular job. Templates help with formatting but do not explain which skills are missing. Reviewing many resumes manually is also time-consuming for placement officers and recruiters.

2.3 Motivation

- Students need to know *why* a resume is weak and *what* to change, not just a template.
- Placement staff cannot review every resume individually.
- Resumes hold personal data, so privacy and role-based access must be built in from the start.
- Automated scores should support human decisions, never replace them.

2.4 Objectives

- **Resume upload and validation** with clear messages for invalid or unreadable files.
- **Information extraction** – education, skills, projects and experience.
- **Analysis and scoring** against defined criteria, with **feedback and improvement suggestions**.
- **Job and internship matching** – matching, missing, required and preferred skills.
- **Placement staff support** through role-based summaries, and **responsible use** of automated results.

2.5 Scope

- **Overall (Sprints 1–7):** user access and upload; analysis and scoring; feedback and explainability; job matching; staff access and reporting; security, privacy and responsible use; maintainability.

- **Sprint 1 (this evaluation):** EPIC 1 – login, resume upload, parsing and invalid-file handling (US-01 to US-04).
- **Out of scope for Sprint 1:** OCR for scanned resumes, AI scoring and skill matching.
- **Deferred to future versions:** job-portal integration, advanced analytics, career roadmap, advanced fairness analysis.

3. Identification of Stakeholders and End Users

Category	Stakeholder	Role and needs
End users	**Students / Job Seekers**	Main users. Upload resumes, need accurate scoring, skill analysis, job matching and clear improvement tips.
	Placement Cell / Officers	Run placement activities, need insights across students and common mistakes.
	Faculty / Career Counsellors	Guide students, need explainable feedback, not just a raw number.
	Recruiters / HR	Screen and shortlist, need reliable skill extraction and job matching.
	Hiring Managers	Make the final call, need trustworthy, unbiased candidate summaries.
Internal / technical	**Development Team**	Build and maintain the system, need clear, feasible requirements.
	UI/UX Designer	Design the upload and results flow, need a usable, accessible design.
	QA / Testers	Verify parsing, scoring and recommendations, need defined test cases.
	System Administrator	Manage accounts, permissions, storage and backups.
	Product Owner / PM	Set priorities and scope across sprints.
	Academic Supervisor	Evaluate the project against course objectives.
Privacy and fairness	**Data Privacy Reviewer**	Ensure resume data is stored and used safely, with consent and retention rules.
	Bias / Fairness Reviewer	Check that scoring and matching do not unfairly favour or reject candidates.
External	**Job Portals** (LinkedIn, Naukri, Indeed)	Supply job and skill data, if integrated, require compliant API use.

Category	Stakeholder	Role and needs
Indirectly affected	**College Management**	Want insight into placement readiness across batches.
	Employer Organisations	Need fair and accurate shortlisting.
	Rejected Candidates	Need a fair process and the possibility of human review.

4. Requirements Elicitation Techniques

Stakeholder	Technique(s)	Why
Students	Survey / Questionnaire, Semi-structured Interviews, Observation, Prototype Evaluation	Reach many students; interviews and observation add detail on real usage.
Placement Officers	Semi-structured Interviews, Document Analysis, Requirements Workshop	Understand the placement process and existing reports.
Faculty / Counsellors	Interviews, Focus Groups, Observation, Prototype Evaluation	Learn what feedback is useful during counselling.
Recruiters / HR	Interviews, Observation, Document Analysis	Identify must-have vs optional skills from job descriptions.
Hiring Managers	Scenario-based Interviews, Document Analysis, Demo Review	See what is needed before a hiring decision.
Development Team	Brainstorming, Requirements Workshop, Interface Analysis	Agree features, boundaries and technical limits.
Academic Supervisor	Document Analysis, Review Meetings	Confirm objectives and evaluation criteria.
Privacy & Fairness Reviewer	Document Analysis (DPDP Act 2023), Scenario-based Elicitation	Identify consent, storage, deletion and fairness rules.
Job Portals	Document Analysis, Interface Analysis	Interviews not possible; study APIs and terms.

Stakeholder	Technique(s)	Why
UI/UX, QA, Sys Admin, PM, indirect parties	Prototyping, Workshops, Interviews, Document Analysis	Usability, testing, operational and reporting needs.

5. Application of Elicitation Techniques

Technique (stakeholder)	Findings so far
Survey + interviews (Students)	Students want a resume score, skill-gap analysis, keyword and formatting tips, and job matching. Prototype evaluation pending.
Interview + documents (Placement Officers)	Officers need batch-level readiness reports and common resume mistakes. Formal interview pending.
Interviews / focus group (Faculty)	Faculty want explainable feedback they can discuss with students, and the option to add comments.
Document analysis (Recruiters)	Job descriptions and ATS matching studied; must-have vs optional skills identified. Interview pending.
Brainstorming + workshop (Dev Team)	Features, system boundary, parsing approach and scoring rules agreed; libraries shortlisted.
Document analysis (Privacy & Fairness)	Consent before upload, secure storage and deletion rules; name, gender, photo and college must not affect the score.
Interface analysis (Job Portals)	Reviewing API access and terms; a local job-description dataset is the fallback.

Student survey (questionnaire and responses): https://github.com/Tanvii13/Resume-Analyzer-for-College-Placements/tree/main/Documents/Survey

5.1 Initial Prioritisation (Requirements Workshop)

Requirement	Priority	Reason
Resume Upload, Parsing, Skill Extraction	Must Have	Needed to process and analyse any resume
Resume Score, Improvement Suggestions	Must Have	Direct benefit to students
Job Matching	Must Have	Supports placement preparation

Requirement	Priority	Reason
Placement Dashboard	Should Have	Useful for placement staff
Advanced Analytics, Portal Integration	Could Have	Can be added later; depends on external APIs

6. Functional Requirements

ID	Functional requirement	Source	Sprint 1 status
FR-01	Allow students to upload a resume.	Survey, Observation	Partially implemented
FR-02	Parse the uploaded resume.	Interview, Brainstorming	Partially implemented
FR-03	Extract relevant skills from the resume.	Survey, Recruiter Interview	Planned
FR-04	Analyse the resume using defined criteria.	Survey, Interview	Planned
FR-05	Generate a resume score.	Survey, Interview	Planned
FR-06	Identify important areas for improvement.	Survey, Observation	Planned
FR-07	Provide improvement suggestions.	Survey, Faculty Interview	Planned
FR-08	Compare resume skills with job requirements.	Recruiter Interview, Documents	Planned
FR-09	Provide suitable job/internship matches.	Survey, Recruiter Interview	Planned
FR-10	Show matching and missing skills for a selected job.	Recruiter Interview	Planned
FR-11	Explain important results.	Faculty Interview, Prototype	Planned
FR-12	Let authorised staff view relevant placement information.	Placement Officer Interview	Planned
FR-13	Provide dashboards/reports for authorised users.	Placement Interview, Workshop	Planned
FR-14	Handle invalid or unsupported resume input.	Observation, Prototype	Implemented

ID	Functional requirement	Source	Sprint 1 status
FR-15	Restrict access to information by user role.	Privacy / Admin Discussion	Partially implemented
FR-16	User login and account access (referenced by US-01).	System Access	Partially implemented

6.1 Domain Requirements

ID	Domain requirement
DR-01	Analysis should focus on information relevant to placement and internship preparation.
DR-02	Job matching should consider the skills and requirements of the target role.
DR-03	Required and preferred skills should be distinguished when the job description specifies them.
DR-04	Related or equivalent skills may need to be considered during matching.
DR-05	Placement eligibility should follow the rules of the placement process.
DR-06	Automated recommendations should support, not replace, human placement or hiring decisions.
DR-07	Student placement information should be accessible only to authorised roles.
DR-08	Resume information should be handled according to institutional privacy rules.
DR-09	Irrelevant candidate characteristics should not unfairly influence scoring or matching.
DR-10	External job information should be used according to the source's rules.

7. Non-Functional Requirements

No measurable targets are defined yet, so most requirements are **specified but not verified**.

ID	Requirement	Status	Sprint 1 evidence
NFR-01	Easy to use for students of all technical levels.	Not verified	Simple upload panel and plain-English errors; no usability test yet.
NFR-02	Results within an acceptable response time.	Not verified	No target defined or measured.
NFR-03	Resume information protected from unauthorised access.	Partially implemented	Hashed passwords, 30-min JWT, private file storage, owner-only access.
NFR-04	Consistent results for the same input and configuration.	Not verified	Applies mainly to scoring (Sprint 2).

ID	Requirement	Status	Sprint 1 evidence
NFR-05	Understandable recommendations and results.	Planned	No recommendations yet.
NFR-06	Clear error messages for invalid input.	Implemented	Stable error codes, covered by tests.
NFR-07	Role-based access control.	Partially implemented	Roles exist; backend endpoints not yet restricted by role.
NFR-08	Readable and accessible interface.	Not verified	No accessibility audit yet.
NFR-09	Avoid unnecessary collection of personal information.	Not verified	No retention or deletion feature yet.
NFR-10	Maintainable parsing, scoring and matching components.	Partially implemented	Backend split into routers and services; two separate parsers exist.
NFR-11	Handle external job-data failures gracefully.	Planned	No external integration yet.

8. User Stories

8.1 Sprint 1 Stories (EPIC 1)

US-01 – User Login and Account Access

Requirement ID(s): FR-16 | *Role:* Student / Job Seeker | *Priority:* Must Have | *Source:* System Access Requirements

Front:

As a student/job seeker, I want to log in to the system, so that I can securely access my resume analysis and related features.

Back (Acceptance Criteria):

1. Given valid login credentials, when the user logs in, then access should be granted.
2. Given invalid credentials, when the user tries to log in, then an appropriate error message should be shown.
3. The system should show features according to the user's role.
4. The user should be able to log out securely.

> **Sprint 1 status:** Partially implemented – works in the Flask frontend (register, login, logout, role tabs). The FastAPI backend has register, login and /auth/me but no logout; the two are not yet connected.

US-02 – Upload Resume

Requirement ID(s): FR-01, NFR-01 | *Role:* Student / Job Seeker | *Priority:* Must Have | *Source:* Survey, Observation

Front:

As a student/job seeker, I want to upload my resume, so that the system can analyse it.

Back (Acceptance Criteria):

1. Given a supported resume file, when the user uploads it, then the system should accept the file.
2. The system should show the upload status clearly.
3. After a successful upload, the user should be able to continue to resume analysis.

> **Sprint 1 status:** Partially implemented – upload works in both components; the backend stores files and records in PostgreSQL, the frontend keeps data in memory.

US-03 – Parse Uploaded Resume

Requirement ID(s): FR-02 | *Role:* Student / Job Seeker | *Priority:* Must Have | *Source:* Interview, Brainstorming

Front:

As a student/job seeker, I want the system to extract information from my resume, so that I do not have to enter the information manually.

Back (Acceptance Criteria):

1. The system should identify information such as education, skills, projects, and experience where available.
2. Extracted information should be organised into understandable sections.
3. The system should indicate information that could not be read or extracted properly.

> **Sprint 1 status:** Partially implemented – the frontend detects sections and contact details; the backend returns plain text only and parse-by-ID fails against the real schema.

US-04 – Handle Invalid Resume

Requirement ID(s): FR-14, NFR-06 | *Role:* Student / Job Seeker | *Priority:* Must Have | *Source:* Observation, Prototype

Front:

As a student/job seeker, I want to know when my resume cannot be processed, so that I can correct the problem and try again.

Back (Acceptance Criteria):

1. The system should reject unsupported, empty, corrupted, or unreadable files.
2. A clear error message should be displayed.

3. The user should be given an option to upload another file.

> **Sprint 1 status:** Implemented – both components reject invalid files with clear error codes; the frontend offers Retry.

8.2 Stories for Later Sprints (Planned)

Story	User story	Acceptance criteria
US-05 Extract Skills from Resume *Must Have*	As a student/job seeker, I want the system to identify skills from my resume, so that I can understand which skills are visible in my resume.	1. The system should identify relevant technical and non-technical skills where possible. 2. Skills should be identified from relevant resume sections. 3. Extracted skills should be displayed clearly to the user.
US-06 Analyse Resume for Placement Preparation *Must Have*	As a student/job seeker, I want my resume to be analysed against placement-related criteria, so that I can understand its strengths and weaknesses.	1. The system should evaluate the resume using defined analysis criteria. 2. The analysis result should be shown clearly. 3. The system should provide consistent results when the same resume and criteria are used.
US-07 Get an Overall Resume Score *Must Have*	As a student/job seeker, I want an overall resume score, so that I can quickly understand how well my resume meets the defined criteria.	1. The system should display an overall resume score. 2. The score should be calculated using defined evaluation criteria. 3. Recalculating the same resume using the same criteria should produce consistent results. 4. The score should be supported by useful information rather than being shown alone.
US-08 Identify Areas That Need Improvement *Must Have*	As a student/job seeker, I want to know which areas of my resume need improvement, so that I can focus on the most important changes.	1. The system should highlight weak or missing areas where applicable. 2. Important improvement areas should be clearly distinguishable. 3. The system should not incorrectly identify a properly included section as a missing or weak area.

Story	User story	Acceptance criteria
US-09 Get Resume Improvement Suggestions *Must Have*	As a student/job seeker, I want suggestions for improving my resume, so that I can make relevant changes before applying for placements.	1. The system should provide suggestions related to identified issues. 2. Suggestions should be clear and understandable. 3. Suggestions should be relevant to the resume content and identified improvement areas.
US-10 Understand the Score and Recommendations *Must Have*	As a student/job seeker, I want to understand why a score or recommendation was given, so that I can decide what changes to make.	1. The system should show the main reasons behind the score or recommendation. 2. Explanations should use simple and understandable language. 3. The system should provide suitable context when an explanation may not be reliable.
US-11 Compare Resume with a Job Description *Must Have*	As a student/job seeker, I want to compare my resume with a job description, so that I can understand how well my profile matches the opportunity.	1. The system should compare resume information with the requirements stated in the job description. 2. Related or equivalent skills should be considered where appropriate. 3. The system should show matching and missing requirements clearly.
US-12 Find Relevant Job or Internship Opportunities *Must Have*	As a student/job seeker, I want to see relevant job or internship opportunities, so that I can identify opportunities that match my profile.	1. The system should display opportunities relevant to the user's profile where data is available. 2. Opportunities should be organised based on their relevance. 3. If no suitable opportunity is found, the system should clearly inform the user.
US-13 View Matching and Missing Skills *Must Have*	As a student/job seeker, I want to see matching and missing skills for a job, so that I can understand what I already have and what I need to improve.	1. Matching and missing skills should be shown separately. 2. Required and preferred skills should be distinguished where the job description provides this information. 3. Related skills should be handled appropriately instead of being treated as completely unrelated skills.

Story	User story	Acceptance criteria
US-14 View Student Resume Analysis *Should Have*	As a placement officer or career counsellor, I want to view student resume analysis, so that I can support students during placement preparation.	1. Only authorised staff should be able to view student analysis. 2. Unauthorised users should not be able to access student resume information. 3. The analysis should be presented in an understandable format.
US-15 View Placement Dashboard and Reports *Should Have*	As a placement officer, I want to view placement-related summaries and reports, so that I can understand overall student placement preparation.	1. The dashboard should provide useful summary information. 2. Sensitive personal information should not be unnecessarily exposed. 3. Reports should follow applicable institutional rules for student data.
US-16 Manage Role-Based Access *Must Have*	As a system administrator, I want to manage role-based access, so that users can access only the features and data allowed for their role.	1. The system should provide different access levels for different roles. 2. Unauthorised users should be prevented from accessing restricted student resume data. 3. Changes to user permissions should be applied correctly.
US-17 Protect Student Resume Data *Must Have*	As a data privacy reviewer, I want student resume data to be protected, so that personal information is handled securely and only when required.	1. Only authorised users should be able to access protected resume information. 2. The system should display only the information necessary for a particular task. 3. Data handling and retention should follow defined privacy rules.
US-18 Keep Automated Results as Supporting Information *Must Have*	As a placement or recruitment decision-maker, I want automated analysis to act as supporting information, so that final decisions remain under human control.	1. The system should not automatically reject a candidate only because of an automated score. 2. Automated results should be presented as supporting information. 3. Final placement or recruitment decisions should remain with the responsible human decision-maker.
US-19 Check Scoring and Matching for Fairness *Must Have*	As a fairness reviewer, I want scoring and matching factors to be reviewable, so that the system can be checked for unfair or unjustified results.	1. The main factors used for scoring and matching should be identifiable. 2. The system should avoid unjustified differences based on characteristics that are not relevant to the job requirements. 3. Potential fairness issues should be recorded for review.

Story	User story	Acceptance criteria
US-20 Keep Core Components Maintainable *Should Have*	As a developer, I want the main system components to be organised separately, so that the system can be updated and maintained more easily.	1. Resume parsing, scoring, and job matching should be organised as separate components. 2. Changes to one component should not unnecessarily affect unrelated features. 3. The code and basic documentation should be understandable for future development.

9. EPICs and Sprint Planning

Sprint / EPIC	Goal	User stories	Status
1. User Access & Resume Upload	Secure access, upload, parsing and invalid-file handling.	US-01 – US-04	In Progress
2. Resume Analysis & Scoring	Extract skills, analyse the resume and give an overall score.	US-05 – US-07	Planned
3. Feedback & Explainability	Improvement areas, suggestions and explanations.	US-08 – US-10	Planned
4. Job / Internship Matching	Compare with job descriptions; matching and missing skills.	US-11 – US-13	Planned
5. Placement Staff Access & Reporting	Staff view of student analysis and summary reports.	US-14, US-15	Planned
6. Security, Privacy & Responsible Use	Role-based access, data protection, human oversight, fairness.	US-16 – US-19	Planned
7. System Quality & Maintainability	Keep components separate and maintainable.	US-20	Planned

Dependency order: Sprint 1 → Sprint 2 → Sprints 3 & 4 (in parallel) → Sprint 5 → Sprint 6 → Sprint 7. Security, privacy, fairness and maintainability are cross-cutting and considered in every sprint.

10. Conflicts Between Requirements

Conflict	Competing needs	Resolution
C-01 Detailed feedback vs. processing time	Detailed explanations for every result can slow the analysis.	Short explanation with the result; more detail on request ("Why this score?").

Conflict	Competing needs	Resolution
C-02 Staff access vs. student privacy	Staff need student analysis; personal data must stay restricted.	Role-based access; individual data only for authorised staff; dashboards show summaries.
C-03 Automation vs. human decisions	Recruiters want automation; the system must not make final decisions.	Results are supporting information; an authorised human decides.
C-04 Desired features vs. scope	Stakeholders want extras; time and resources are limited.	Core features first; dashboards lower priority; extras in future versions.
C-05 Candidate filtering vs. human review	Strict filtering is faster but may hide candidates from review.	Scores organise results but never permanently remove a candidate.
C-06 Analytics vs. data retention	Trends need history; resume data should not be kept too long.	Follow institutional retention rules; use aggregated data for long-term reports.
C-07 Simple score vs. detailed evaluation	Students want one number; counsellors need detail.	One overall score first, with a breakdown below it.
C-08 AI assistance vs. requirement accuracy	AI drafts may include assumptions and raise privacy concerns.	Treat AI output as a draft; check against sources; anonymise data.
C-09 AI-assisted coding vs. understanding	Generated code may contain errors or not be understood.	Review and test generated code; the author must understand it.
C-10 Fast development vs. maintainable design	One big component is faster now but harder to change.	Keep processing, extraction, scoring, matching and UI separate.

11. Proof of Concept (POC) – Sprint 1

11.1 Objective

Sprint 1 delivers EPIC 1 – **User Access & Resume Upload**: a student can register, log in, upload a PDF or DOCX resume, have its content read, and get a clear message with a retry option when the file cannot be processed (US-01 to US-04).

Sprint 1 was built as two components: a **Flask frontend prototype** with the full user flow (using temporary in-memory storage), and a **FastAPI backend** with a **PostgreSQL** schema for authentication, persistent uploads and parsing.

> **Integration pending:** The Flask frontend does not yet call the FastAPI backend or use PostgreSQL. Connecting them is the main task left for EPIC 1.

11.2 Scope of Sprint 1

Frontend (Flask + Jinja2, HTML, CSS, JavaScript; designed in Figma)

- **Pages:** Login (Student / Placement Officer / Admin tabs), Register (always creates a Student account), Dashboard (drag-and-drop upload, checklist, resume history with Retry), Parsed Resume view, 403/404 error pages.
- **Features:** form validation, hashed passwords (Werkzeug), HttpOnly session cookie, files saved privately under random names, only students can upload, users see only their own resumes.
- **Parser:** validates the file, extracts text with pypdf / python-docx, splits it into sections (Education, Skills, Projects, Experience, etc.), extracts contact details and lists missing key sections.

Backend (FastAPI + Uvicorn)

- **Authentication:** register (Argon2 hashing, student role assigned), login (JWT, HS256, 30 minutes), current user.
- **Resumes:** upload with validation and storage, direct parse, and parse of a stored resume by ID.
- **Parser:** checks extension, size, file signature, ZIP/PDF structure and encryption; extracts PDF pages and DOCX headers, paragraphs, content controls and tables; returns stable error codes.

Database (PostgreSQL via psycopg 3)

Tables `users`, `roles`, `user_roles`, `resumes` and `resume_sections` with UUID keys, foreign keys and CHECK constraints; roles `student`, `placement_officer` and `admin` are seeded. Files are stored outside the database and referenced by `storage_key`.

11.3 API Endpoints

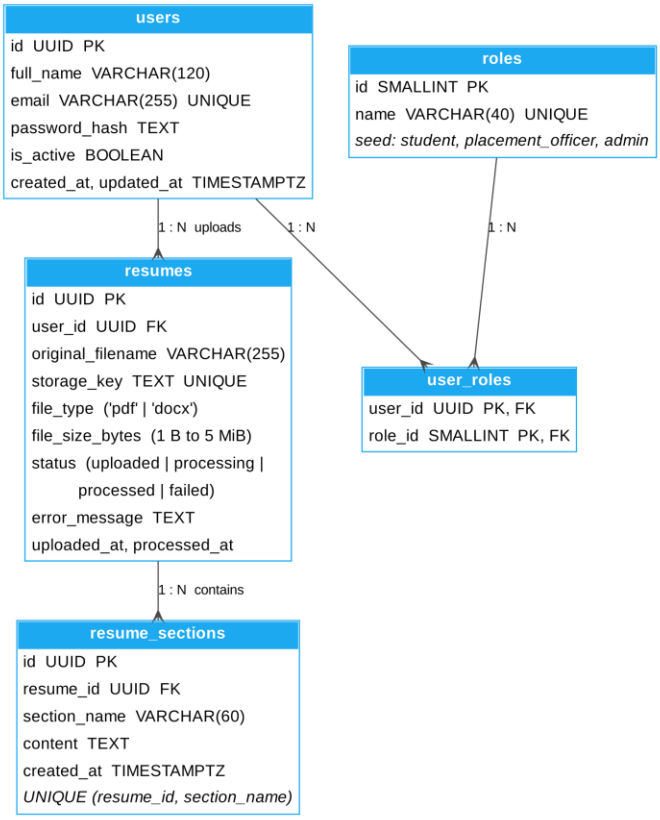
Endpoint	Auth	Purpose	Main responses
`POST /auth/register`	–	Create a student account	201; 409 duplicate; 422 invalid
`POST /auth/login`	–	Return a JWT access token	200; 401 invalid credentials
`GET /auth/me`	Bearer	Current user's email and role	200; 401
`POST /resumes/upload`	Bearer	Validate, store and record a resume	200; 400 / 413 / 415 with error code
`POST /resumes/parse`	Bearer	Validate and parse without storing	200 text and counts; 400 / 422

Endpoint	Auth	Purpose	Main responses
`POST /resumes/{id}/parse`	Bearer	Parse a stored resume and update its status	200; 404; 500 (defect D-1)

12. Technology Stack

Layer	Technology	Role
Frontend	Flask, Jinja2, HTML, CSS, JavaScript	Server-rendered pages, routing, sessions
UI design	Figma	Screen designs
Backend	FastAPI, Uvicorn, Pydantic	REST API and validation
Authentication	pwdlib (Argon2), PyJWT	Password hashing, bearer tokens
Resume processing	pypdf, python-docx	Validation and text extraction
Database	PostgreSQL, psycopg 3	Users, roles, resume metadata
Testing	pytest, FastAPI TestClient (httpx), Flask test client	Automated tests
Version control	Git, GitHub	Collaboration
NLP / scoring / deployment	Not yet chosen	Planned for later sprints

12.2 Database Design



13. Contributions

Name	Student ID	Work done
Sujal Balva	202401216	Stakeholders and end users, database development.
Tanishk Dhawan	202401224	Elicitation techniques, requirement conflicts.
Topaliya Shlok Ashokbhai	202401229	Elicitation techniques, user stories, backend development.
Trivedi Atharv	202401230	Elicitation techniques, requirement conflicts, AI/ML and NLP.
Vamja Krish Pravinbhai	202401239	
Nakum Tanvi Dhiren	202401262	Stakeholders, EPICs, concept poster, Figma UI design, frontend development.

Name	Student ID	Work done
Solanki Taniskumar Mehulbhai	202401264	Applied elicitation techniques, user stories, backend development.
Tejot Harshil Rajendrabhai	202401265	Applied elicitation techniques, EPICs, Figma UI design, frontend development.
Krish Hitendrakumar Paghadar	202401429	Stakeholders and end users, frontend development.
Prerak Patel	202401452	Backend development.

13.1 Sprint 1 Development Teams

- **Frontend:** Tanvi Nakum, Harshil Tejot, Krish Paghadar
- **Backend:** Tanish Solanki, Shlok Topaliya, Prerak Patel
- **Database:** Sujal Balva
- **AI/ML and NLP:** Atharv Trivedi

14. Conclusion

The Resume Analyser for College Placements helps students understand and improve their resumes through validated upload, information extraction, an explained score, improvement suggestions and job matching, while keeping automated results as support for human decisions.

The requirements work is complete: stakeholders, elicitation, 16 functional, 11 non-functional and 10 domain requirements, 20 user stories, 7 EPICs and 10 resolved conflicts. In Sprint 1, a Flask prototype delivers the full register → upload → parse → retry flow, and a FastAPI backend with PostgreSQL provides authentication, persistent uploads and a well-tested parser; all 90 automated tests pass. Integrating the two components and fixing the two defects are the next steps before Sprint 2.